

```
# Extensions processing must not fail
if not extensions_are_valid(acl, acl_entry, subject_desc, endpoint_id,
cluster_id): continue

# All checks have passed, add privilege to granted privilege set
add_granted_privilege(granted_privileges, acl_entry.privilege)

# Should never grant Administer privilege to a Group.
if subject_desc.AuthMode == AuthModeEnum.Group:
    assert (PrivilegeEnum.Administer not in granted_privileges)

return granted_privileges

# Note that `arl` is either the `arl_attr` or `commissioning_arl_attr`.
def is_request_restricted(arl, subject_desc, request_type, endpoint_id, cluster_id,
cluster_element):
    # Handles ensuring that only endpoints and clusters for which access restrictions
    are allowed
    # are subject to the restriction. Only mandatory clusters on endpoints with device
    type allowing
    # Managed Device ACL cluster feature can be restricted. Some specific clusters and
    elements can
    # never be restricted. See the Managed Device feature section of the ACL cluster for
    details.
    if are_restrictions_disallowed(request_type, endpoint_id, cluster_id,
cluster_element):
        return False

    for arl_entry in arl:
        if not subject_desc.IsCommissioning:
            if arl_entry.FabricIndex != subject_desc.FabricIndex: continue

        if arl_entry.Endpoint != endpoint_id: continue

        if arl_entry.Cluster != cluster_id: continue

        for arl_restriction in arl_entry.Restrictions:
            # if the restriction is attribute access
            if arl_restriction.Type == AccessRestrictionTypeEnum.AttributeAccessForbidden
and request_type in [RequestTypeEnum.AttributeRead, RequestTypeEnum.AttributeWrite]:
                # if the attribute id is NULL (wildcard) or matches the cluster_element
                if arl_restriction.ID == NULL or arl_restriction.ID == cluster_element:
                    # then no granted privileges
                    return True
```